Program: Monte Carlo Simulation

Description: Monte Carlo Simulation is a statistical method used to predict the behaviour of systems involving a degree of uncertainty. This technique estimates possible outcomes and their probabilities by running many simulations with random inputs.

Inputs:

- N (how many i.i.d random variables)
- Which kind of random variable (give them choices of the common distributions), parameters for each. Common distributions and their parameters (floats):
    - Normal distribution: std, mean
    - Uniform distribution: min, max
    - Exponential distribution: lambda
    - Bernoulli: p
    - Binomial: n (int), p
    - Poisson: lambda
- What the user want to estimate, such as a function f(x, y) = x*y^2
    - (the formula will be parsed in the program to use for simulation)

Process:

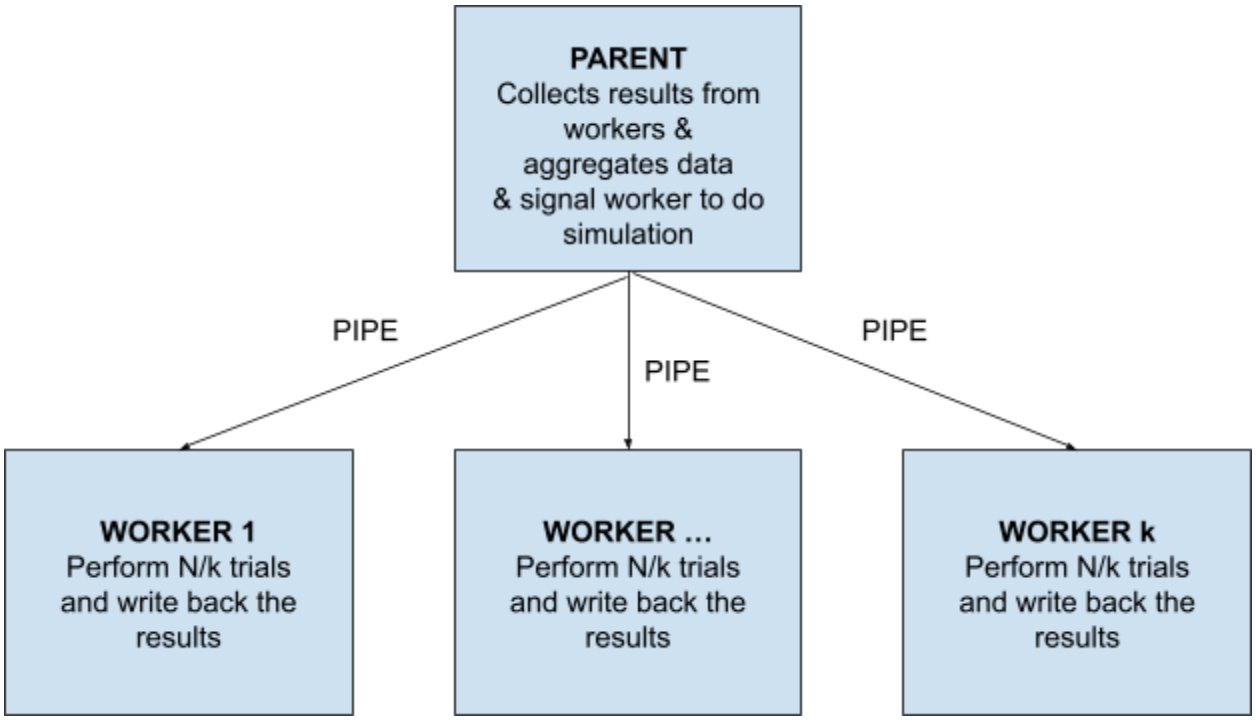
1. The user is prompted to enter the function they want to estimate, the variables involved, and the distributions for each variable, including any necessary parameters as listed above depending on what distribution they chose. They are also asked to provide the total N number of simulations to run.
2. The parent process first parses the function and distributions, then creates a fixed number of worker processes (the worker pool). Each worker repeatedly receives a task from the parent, which consists of generating a chunk of samples. For each sample, the worker draws values from the specified distributions, evaluates the function given by the user using those values, and accumulates a local sum.
3. After completing its assigned chunk, the worker sends its partial result back to the parent through a pipe. The parent collects these partial results, aggregates them (using total_sum and total_samples), and, if more simulations remain, sends the next chunk of work to the idle workers.
4. Once all simulations are complete and all workers have returned their results, the parent computes the final estimate of the expected value by dividing the total sum by the total number of samples and returns the result to the user.

Architecture: We decided to use a **worker pool** instead of parallel forking because using a fixed worker pool is more efficient and safer than spawning a new process for each simulation or chunk. Creating processes is expensive in terms of memory and CPU resources, and spawning many processes in parallel can overwhelm the system. A worker pool allows the same processes to be reused across all tasks, keeping resource usage predictable while still achieving parallelism. It also simplifies task management aggregation of results.

Output: Estimate expected value of the function that the user specifies.

Summary: The user first selects a probability distribution and provides the required parameters as well as the function they want an estimate of. The parent process then creates multiple child processes to perform Monte Carlo simulations using the function. Each child generates random samples from the chosen distribution, computes partial statistics from its simulations, and sends the results back to the parent process through a pipe. The parent collects the results from all child processes, aggregates the data, and computes the final estimate of the expected value.

An initial sketch of the communication protocols:



Message 1: Worker telling parent result of simulation

Field	Description
Direction	Worker -> Parent
Encoding	A struct containing the result of the simulation(s) the child ran: typedef struct { long trials; double sum; double sum_sq; } Result;
Semantics	Every worker will run its own simulation(s) according to the message it received, and upon completion, writes its result back to the parent.
Error Handling	If write returns -1, the parent logs the error, marks the child as dead, and creates a replacement child which will attempt a simulation again (or a less complex option would be to just print error and terminate worker).

Message 2: Parent signaling worker to do simulation (task)

Field	Description
Direction	Parent -> Worker
Encoding	A struct containing a chunk of simulations to perform: typedef struct { long num_samples string distribution float parameter1 float parameter2 } Task;
Semantics	Parent sends each worker a task describing how many samples to generate and the distributions to sample from. Worker executes the simulations and returns a Result message when done.
Error Handling	If write fails, the parent logs the error and may either resend the task to the same worker or mark the worker as dead and assign the task to another worker.